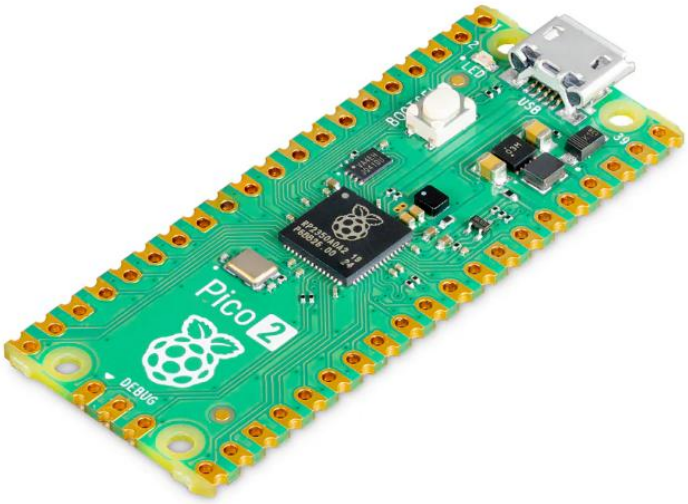


PISYM HARDWARE

ABSTRACT

PiSYM offers the ability to add hardware to replicate the SYM-1's capabilities natively.

Brian Campbell

Contents

1 What is PiSYM?

The PiSYM is a Raspberry Pi Pico 2 based emulator for the Synertek Systems SYM-1 computer of the 1970s. PiSYM emulates a SYM-1 with the full 32K of RAM with both the BASIC and RAE ROMs installed.

Importantly, PiSYM runs at the same speed as a real SYM-1 down to the instruction level. A two-cycle instruction on a 1MHz 6502 will execute in 2µs. That same instruction on a PiSYM will also execute in 2µs. This is essential as many time-critical functions in the SYM-1 are based on timing loops.

Address aliasing is also replicated in PiSYM. Supermon occupies a 4K block beginning at $8000 but due to the wiring of the address bus, is aliased starting at $F000 so the interrupt vectors are at the correct location. Similarly, the 128 byte system RAM that occupies $A600 to $A67F is aliased at $A680 to $A6FF, $A700 to $A77F, and $A780 to $A7FF. If you change a value at address $A600, it will appear at $A680, $A700, and $A780 as well.

The goal of the PiSYM is that it should be impossible to differentiate between it and a real SYM-1 from within the environment for the given hardware installed. In addition to the RAM and ROM, PiSYM also emulates the SY6532 RIOT chip that provides system RAM, I/O, and a Timer. Even 6532 registers are emulated with the DDR and data registers operating the Raspberry Pi Pico 2 GPIO pins.

PiSYM runs Supermon V1.1, BASIC, and RAE unmodified but also has the ability to interwork with more modern peripherals. While PiSYM retains the SYM-1's capability to communicate with a CRT via RS232 or TTY via a 20mA loop, it can also communicate with a terminal via USB. Although PiSYM can save and load data via a cassette recorder, it also has the ability to store and retrieve programs from the internal flash storage of Raspberry Pi Pico 2 or an external SD card. Like the USB port, PiSYM does this transparently to the SYM-1 software such that it thinks it is saving and loading from cassette tape. This even translates to the LOAD and SAVE commands in BASIC.

PiSYM also contains its own debugger. While the SYM-1 has its own debugger and this is emulated along with everything else, this debugger is primitive by modern day standards. The PiSYM debugger handles more advanced features such as breakpoints and watches and unlike the SYM-1 debugger, is capable of debugging Supermon itself.

2 Hardware Options

At its most basic level, PiSYM requires nothing more that the Raspberry Pi Pico 2 itself. Once PiSYM is installed, simply connect the USB port to a PC and use a terminal program such as PuTTY to connect. It will behave exactly like a SYM-1 would if connected to a CRT. If this is all the functionality that is desired, then that is all that is required. A SYM-1 can be had for the price of a Raspberry Pi Pico 2.

In order to experience additional capabilities, additional hardware can be added. Most of that hardware is basically the same as that found on an actual SYM-1. A display and keypad can be added to replicate the six-digit display and keyboard input of the original. With a small amount of additional circuitry, the actual CRT and TTY interfaces can be added.

Some options are unique to the PiSYM such as the external SD card.

2.1 PiSYM I/O Layout

PiSYM uses virtually all of the I/O ports on the Raspberry Pi Raspberry Pi Pico 2.

Raspberry Pi Pico 2 Pin	Name	PiSYM Name	Function
1	GP0	6532 PA0	Used to connect to seven segment display, membrane keypad, speaker, and audio output. Although theSYM-1 used common anode seven segment display digits, the circuit can be modified to use common cathode displays without any impact on the Supermon program. The GP pins on the Raspberry Pi Pico 2 are not able to drive the segments, so an interface chip is required. Be wary of I_O and I_{VCC} current limits.
2	GP1	6532 PA1	
3	GND		
4	GP2	6532 PA2	
5	GP3	6532 PA3	
6	GP4	6532 PA4	
7	GP5	6532 PA5	
8	GND		
9	GP6	6532 PA6	
10	GP7	6532 PA7	
11	GP8	6532 PB0	
12	GP9	6532 PB1	
13	GND		
14	GP10	6532 PB2	
15	GP11	6532 PB3	
16	GP12	6532 PB4	
17	GP13	6532 PB5	
18	GND		
19	GP14	6532 PB6	
20	GP15	6532 PB7	
21	SPIO RX	MISO	Interface to SD Card.
22	SPIO CSn	CS	
23	GND		
24	SPIO SCK	SCK	
25	SPIO TX	MOSI	
26	GP20	Reserved	
27	GP21	Reserved	
28	GND		
29	GP22	SYNC	Provides a waveform indicating emulation performance.
30	RUN	~PiSYM RESET	Connect to GND via push button to reset Raspberry Pi Pico 2.
31	GP26	~IRQ	Simulates IRQ pin on 6502
32	GP27	~NMI	Simulates NMI pin on 6502. Connects to SYM-1 DEBUG ON/OFF RS flip-flop
33	GND		
34	GP28	~RESET	Simulates RESET pin on 6502
35	ADC VREF		
36	3V3 (OUT)		
37	3V3 EN		
38	GND		
39	VSYS		Used for external 5V in via diode
40	VBUS		

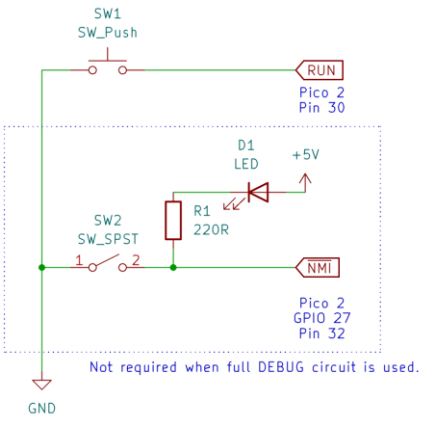
2.2 PiSYM Debugger Access

Normally, PiSYM will boot exactly as a SYM-1 does. Alternatively, it can boot into PiSYM's debugger, not to be confused with Supermon's debugger.

When PiSYM starts, it looks at GPIO27 that it uses to simulate the NMI pin on a 6502. If it is low, PiSYM boots to its debugger.

In addition, when running a terminal emulator connected to the USB port, the F1 key will cause invoke the PiSYM command line where the SYM-1's keyboard input function is called.

2.3 Raspberry Pi Pico 2 RESET and Minimal Debugger



While a bare Raspberry Pi Pico 2 can be used, its use can be made easier by the addition of at least a pair of switches.

A momentary action switch grounding the Raspberry Pi Pico 2 RUN pin (pin 30) resets the Raspberry Pi Pico 2 itself.

Normally a pair of buttons on the SYM-1 keypad operates a RS flip-flop connected to the NMI pin to actiavte the debugger. If the full keypad is not implemented, single pole, single throw switch attached to the emulated NMI pin (GPIO 27, pin 32) will achieve the same result.

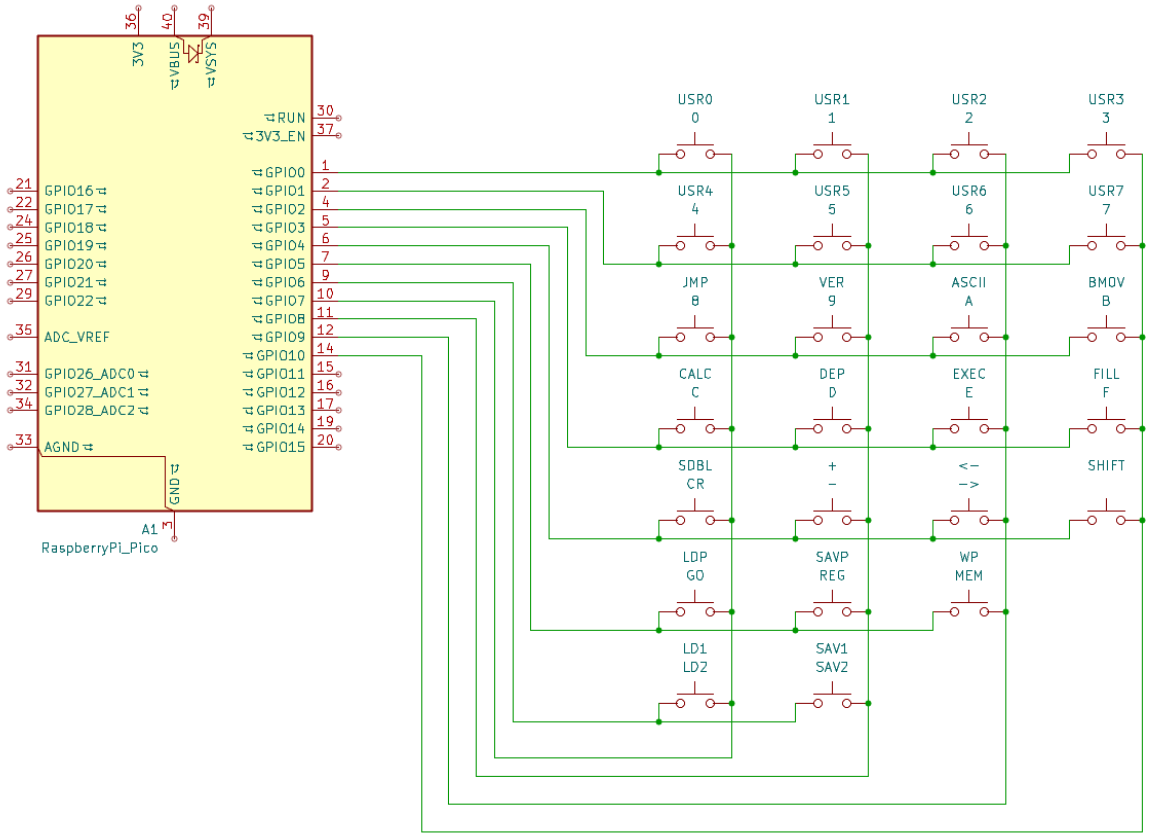
Not only does the DEBUG switch activate the SYM-1 debugger, it also makes PiSYM boot to the command line when it is reset by momentarily grounding the RUN pin.

2.4 Seven Segment Display and Keypad

The way the SYM-1 works, use of the keypad and the seven-segment display is linked, so both should be implemented together.

The keypad is implemented by installing switches between the I/O pins of the 6532 RIOT chip as per the SYM-1 circuit diagram.

2.4.1 Keypad



Note that the RST and DEBUG buttons are not included. These are connected to alternative circuitry described below.

In the original schematic, the A-17, A-Z, A-X, and A-V blocks simply refer to external connections on the SYM-1 circuit board.

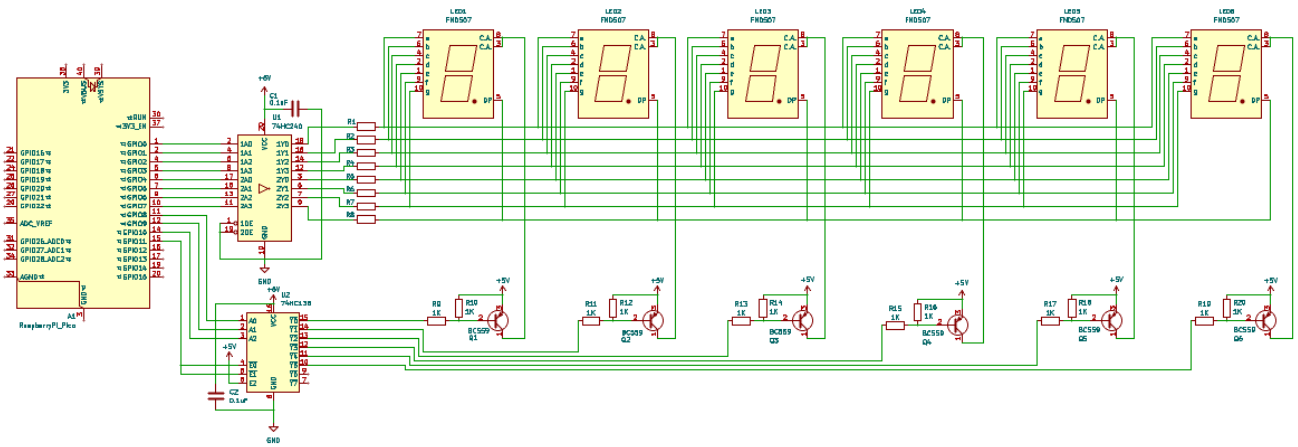
PA0-7 of the SY6532 map to GPIO0-7 of the Raspberry Pi Pico 2. Similarly, PB0-7 of the SY6532 map to GPIO8-15 of the Raspberry Pi Pico 2.

The real SYM-1 used a membrane keypad. This can be matched using an array of tactile switches, a 3D printed frame, and a flexible print of the membrane image. The switch spacing, frame, and membrane can be scaled to match any size you want.

2.4.2 Seven Segment Display

The SYM-1 seven segment display uses common anode digits. While you can replicate this circuit, an alternative for common cathode digits is provided. To the Raspberry Pi Pico 2 and SYM software, it simply cannot tell.

2.4.2.1 Common Anode



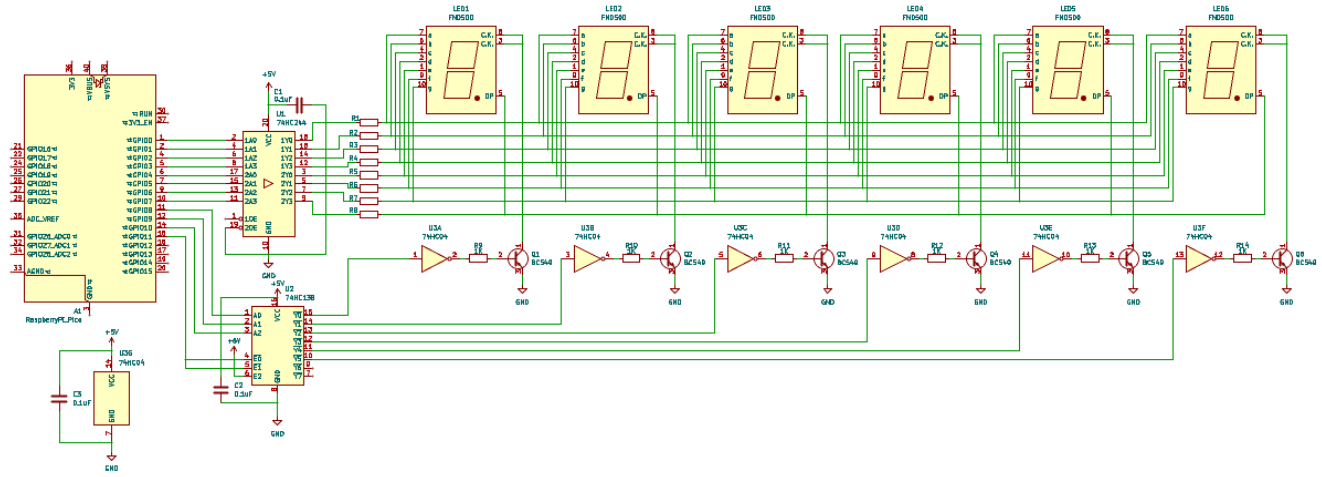
The display circuit shares the same I/O pins as the keyboard. The SYM-1 software is able to handle this natively.

The original SYM-1 circuit used a 74145 BCD to decimal decoder. The HC equivalent may be difficult to obtain. A more readily available 74HC138 can be used in its place. Only one function is lost due to this substitution related to the speaker, but that is easily handled via a small external circuit.

A 74HC240 is used to drive the segments. It substitutes the old 7416 used on the standard SYM-1 as the latter chip may be difficult to find.

R1 to R8 are notionally 150Ω resistors, however the 74HC240 will then be pushed over its limits regarding the amount of current dissipated through its ground pin. The multiplexing and alternating use with the keypad will reduce the average load.

2.4.2.2 Common Cathode

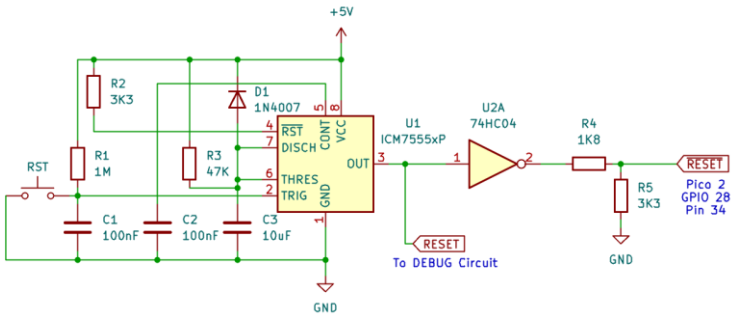


The common cathode version is similar to the common anode version. A 74HC244 is used instead of the 74HC240. It is the same chip but with non-inverted outputs instead of the 74HC240's inverted outputs.

The 74HC138 drives an inverter that is used to switch on NPN transistors connected to the common cathode of the LED displays.

The same issues exist regarding the loading of the segment driver chip. In practice, this circuit works with no issues.

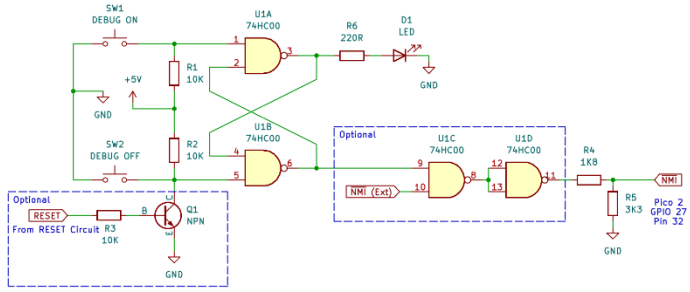
2.4.3 Reset



If you wish to replicate the SYM-1 reset function, the RST button connects to a 555 timer circuit that generates a low signal of roughly 0.5s duration. This output connects to GPIO28 (pin 34) which emulates the 6502 RESET pin.

The circuit is a fairly standard 555 circuit. The 1MΩ resistor and 0.01µF capacitor holds the 555 trigger pin (pin 2) holds the trigger low on power up to deliver a power-on reset. The triple input NOR gate (U3) merely inverts the output, so any equivalent built from a NAND or NOR gate or an inverter can be used. Also, it's fine to use a 74HC series device rather than the old 74LS series. Finally, given the Raspberry Pi Pico 2 inputs are 5V tolerant, this external circuitry can run on 5V if 3.3V is not adequate.

2.4.4 Debug



The DEBUG ON and OFF buttons operate an RS flip-flop built from two NAND gates. The NAND gates can be created from the more common 74HC00 chips rather than the ancient 7400. The ON and OFF buttons connect their respecting inputs to ground.

In the SYM-1, there is a link from the output enable pin of the Supermon ROM that disables NMI while running Supermon code as this is where the debugging software resides. In PiSYM, there is no requirement to duplicate this link as it is handled within the firmware.

There are two options with this part of the circuit. A transistor can be used to pull down the DEBUG OFF line when the 6502 Reset 555 timer is active. This ensures that the processor boots with DEBUG OFF as it does in the SYM-1. Another option is to use the two remaining gates on the 74HC00 to provide external access to the NMI signal in parallel with the DEBUG flip flop. Regardless of the state of the DEBUG flip flop, pulling the NMI (Ext) line low sends an immediate NMI signal to the 6502.